



DOCUMENTATION PRODUIT

Manuel détaillé des écrans

Description enrichie — UI · données · actions · états · navigation · accès

53 écrans

14 zones · audit du code source réel (lecture seule)
28 juin 2026

Description **enrichie** des 53 écrans ([src/features/](#)). Pour chacun : **Rôle & contexte** · **UI** · **Données (hooks/API/stores)** · **Actions** · **États** · **Navigation** · **Accès**.

Basé sur l'audit du code source réel (lecture seule) · 28 juin 2026.

Conventions techniques transverses

- **Réseau** : tout passe par `apiClient` (baseURL `/api`, enveloppe `Envelope<T>`). Les modules « core » dé-wrappent `res.data.data` ; les modules `extensions/api/*` consomment du JSON brut.
- **Cache/serveur d'état** : TanStack Query (clés centralisées `roomKeys`, `houseKeys`, `profileKeys`, `notificationKeys`...). Mutations souvent optimistes avec rollback.
- **Temps réel** : Socket.IO singleton, activé seulement si authentifié et `env.REALTIME_ENABLED` (sinon polling/mocks). Hooks `useRoomSocket`, `useChatSocket`, `useGroupSocket`, `useHallwaySocket`, `useNotificationSocket`, `useTypingIndicator`.
- **Vocabulaire** : « club » (backend) = « house » (front). Mapping privacy MAJ→min dans `houseService`.
- **Rôles** : `USER(0)` < `MODERATOR(1)` < `ADMIN(2)` < `SUPER_ADMIN(3)` (`isAtLeast`). Les écrans admin masquent l'UI selon le rôle ; **chaque endpoint** `/admin/*` **re-vérifie côté serveur**.
- **Commun** : i18n `react-i18next`, thème NativeWind (Material 3 Dark), `safe-area-context`.

1. Authentification (6)

1. LandingScreen

- **Rôle & contexte** : Accueil non authentifié (route initiale de `AuthNavigator`, après `WelcomeSlides`). Fond dégradé animé.
- **UI** : `GradientView` + cercles déco, logo `mic` + tagline, 3 `FeatureItem` (rooms/houses/chat), `AvatarsPreview` (7 avatars), `LandingCTA` (« Get started », « Login », lien « dev skip » si `__DEV__`). Animations reanimated.
- **Données** : `useAuthStore` (`devLogin`, `status`). Aucun réseau hors devLogin.
- **Actions** : « Get started » / « Login » → `navigate('Phone')` ; dev skip → `devLogin()` (`POST /auth/dev-login`).
- **États** : `devSkipPending` lié à `status==='authenticating'` (« ... »).
- **Navigation** : → Phone (devLogin court-circuite vers Main).
- **Accès** : non authentifié.

2. PhoneScreen

- **Rôle & contexte** : Saisie du numéro + confirmation d'âge pour demander l'OTP.
- **UI** : `Input` téléphone (autoFocus, adornement sélecteur pays `flag` + indicatif → `CountryPicker`), checkbox « ≥ 16 ans », `Button` « Next », liens Terms/Privacy. Formatage live `libphonenumber-js`.
- **Données** : `react-hook-form` + `zodResolver(phoneFormSchema)` (E.164, `ageConfirmed`). Pays via `react-native-localize`. `useAuthStore.requestOtp`.
- **Actions** : submit → `requestOtp(phone)` (`POST /auth/send-otp`) → `navigate('Otp',{phoneNumber})` ; liens → Terms / PrivacyPolicy.
- **États** : `isValid` gate le bouton ; `isSubmitting` → loading ; erreurs API mappées au champ.
- **Navigation** : Landing → Otp / Terms / PrivacyPolicy.
- **Accès** : non authentifié.

3. OtpScreen

- **Rôle & contexte** : Vérification du code OTP 6 chiffres.
- **UI** : numéro masqué, `OtpInput` 6 cellules (autoFocus, shake reanimated sur erreur), message tentatives restantes, bloc resend (bouton ou compte à rebours).
- **Données** : `route.params.phoneNumber` ; `useAuthStore` (`verifyOtp`, `requestOtp`) ; state local `code/error/attempts/countdown`.
- **Actions** : auto-submit à 6 chiffres → `verifyOtp` (`POST /auth/verify-otp`, renvoie `{isNewUser}`) → nouveau : `navigate('Name')` (existant : bascule app) ; resend → `requestOtp` + reset.
- **États** : `MAX_ATTEMPTS=5` → `locked` ; `RESEND_COOLDOWN=60s` ; `isSubmitting` → « verifying » ; nouveaux comptes restent `authenticating`.
- **Navigation** : Phone → Name (nouveau) / app (existant).
- **Accès** : en cours d'authentification.

4. NameScreen

- **Rôle & contexte** : Vrai nom (prénom + nom) avant le username (ordre Clubhouse).
- **UI** : 2 `Input` (First name autoFocus, Last name, max 50), `Button` « Next ».
- **Données** : `useOnboardingStore.setProfile` (stash local, flush ultérieur à `completeOnboarding`). Aucun réseau.
- **Actions** : « Next » → `setProfile({firstName,lastName})` → `navigate('Username')`.
- **États** : `canContinue = firstName non vide` (lastName optionnel).
- **Navigation** : Otp → Username.
- **Accès** : nouvel utilisateur, statut `authenticating`.

5. UsernameScreen

- **Rôle & contexte** : Choix du `@username` ; promeut le compte en `authenticated`.
- **UI** : `Input` username (adornement `@`, compteur), jusqu'à 3 puces de suggestions, `Button` « Submit ».
- **Données** : `authService.suggestUsername()` (`GET /users/suggest-username`) ; `react-hook-form` + `zodResolver(usernameFormSchema)` (3-24, `[a-z0-9_]`) ; `useAuthStore.setUsername`.

- **Actions** : tap suggestion → remplit ; submit → `setUsername` (`PATCH /users/me/username`) → `authenticated` .
- **États** : `loadingSuggestions` ; `isValid` gate ; erreurs validation/API mappées.
- **Navigation** : Name → (RootNavigator bascule vers Onboarding ou Main).
- **Accès** : nouvel utilisateur, `authenticating` .

6. WaitlistScreen

- **Rôle & contexte** : Écran « liste d'attente » + invitation virale. **Enregistré dans Auth mais jamais routé** (pas de gating réel).
 - **UI** : icône `hourglass-empty` , titre/sous-titre, boutons « Invite a friend » + « Back ». Écran statique.
 - **Données** : aucune (i18n seul).
 - **Actions** : « Invite a friend » → `Share.share` ; « Back » → `goBack` .
 - **États** : aucun.
 - **Navigation** : deep-link `auth/waitlist` uniquement (inatteignable par le flux).
 - **Accès** : non authentifié.
-

2. Onboarding (4)

7. WelcomeSlidesScreen

- **Rôle & contexte** : Carrousel pédagogique pré-auth, affiché **une seule fois** (4 slides : welcome/rooms/clubs/topics).
- **UI** : `FlatList` horizontale `pagingEnabled` (icône + titre + corps), dots de progression, « Skip » (caché au dernier), `Button` « Next » / « Get started ».
- **Données** : `welcomeStorage` (`AsyncStorage` `chathouse.welcomeSlides.completed.v1`). Aucun réseau.
- **Actions** : Next → slide suivant / `goLanding` ; Skip / Get started → `markSeen()` + `replace('Landing')`.
- **États** : pas de loading ; `isLast` masque Skip.
- **Navigation** : `AuthNavigator` (route initiale résolue par `welcomeStorage.hasSeen()`) → Landing.
- **Accès** : non authentifié, premier lancement.

8. SetupProfileScreen

- **Rôle & contexte** : Étape 1 onboarding post-auth : photo + nom affiché + bio.
- **UI** : avatar `Pressable` (preview/ `camera-alt`), bouton « Import from X » (si `twitter.configured`), `Input` `displayName` (max 60) / bio (max 150), `Button` « Continue » + « Skip ».
- **Données** : `useOnboardingStore.setProfile` ; `useTwitterImport` → `twitterApi` (`GET /ext/twitter/status` , `begin` , `complete` via deep-link `chathouse://oauth/twitter`) ; `mediaService.uploadAvatar` (`POST /upload/avatar`) ; `launchImageLibrary` ; `setupProfileFormSchema` .
- **Actions** : pick image ; Import X → remplit `displayName` / bio / avatar ; Continue → upload avatar puis `setProfile` → `navigate('InterestSelection')` ; Skip → idem sans persister.
- **États** : `uploading` / `isSubmitting` / `twitter.importing` ; erreurs zod + `Alert` .
- **Navigation** : `OnboardingNavigator` (route `Onboarding`) → `InterestSelection`.
- **Accès** : authentifié avec `hasCompletedOnboarding===false` .

9. InterestSelectionScreen

- **Rôle & contexte** : Sélection des centres d'intérêt (chips), **min 3 / max 10**.
- **UI** : hint dynamique, `ScrollView` de chips toggle, `Button` « Finish » désactivé < 3.
- **Données** : `useOnboardingStore.setInterests` ; liste statique `INTEREST_CATEGORIES` . Aucun réseau.
- **Actions** : toggle chip (cap 10) ; Finish → `setInterests` → `navigate('NotificationsPermission')` .
- **États** : `canSubmit` = `≥ 3` ; cap silencieux à 10.
- **Navigation** : `SetupProfile` → `NotificationsPermission`.
- **Accès** : onboarding en cours.

10. NotificationsPermissionScreen

- **Rôle & contexte** : Demande la permission push (« Turn on notifications »).
 - **UI** : icône `notifications-active` , 3 bénéfices, `Button` « Enable » + lien « Not now ».
 - **Données** : `pushService.registerWithBackend()` (permission OS → `POST /push/register` , idempotent).
 - **Actions** : Enable → `registerWithBackend()` (erreurs avalées) → `navigate('SuggestedFollows')` ; Not now → idem.
 - **États** : `requesting` (loading) ; aucun blocage sur erreur.
 - **Navigation** : `InterestSelection` → `SuggestedFollows`.
 - **Accès** : onboarding en cours.
-

3. Hallway & Salons audio (5) — cœur de l'app

11. RoomFeedScreen

- **Rôle & contexte** : Accueil du RoomsTab (« hallway ») : feed des rooms live rankées + upcoming + strips d'extensions.
- **UI** : Header (logo + search/event/play-circle/notifications [badge]) ; `FilterPill` (All/Following/Clubs/Tech/Music/Business/Health) ; strips ext (`ExtAvailablePeopleStrip`, `ExtSuggestedFollowsStrip`, `ExtUpcomingForYouStrip`, `ExtRecentlyPlayedStrip`) ; `UpcomingRow` ; `FlatList` de `RoomCard` (catégorie, house, amis inside, `ParticipantsRow`, compteurs, Join) ; FAB « + ».
- **Données** : `useRooms(filter)` → `useInfiniteQuery` `GET /rooms/feed` ; `useQuery` upcoming `GET /rooms?filter=upcoming` ; `useHallwaySocket()` (invalide sur new/ended) ; `useUnreadNotificationCount()` ; `useExtWave()`.
- **Actions** : filtre → refetch ; cartes/strips → `navigate('Room',{roomId})` ; FAB → `CreateRoom` ; icônes → Explore/Events/Replays/Notifications ; wave ; pull-to-refresh + `fetchNextPage`.
- **États** : `RoomFeedSkeleton` (loading) ; `EmptyState` (error) ; `keepPreviousData` (pas de flash au filtre) ; temps réel socket.
- **Navigation** : tab par défaut → Room, CreateRoom, Explore, Events, Replays, Notifications.
- **Accès** : authentifié.

12. RoomScreen

- **Rôle & contexte** : Salle audio live (orchestrateur ~930 lignes) : scène/écoute, LiveKit, rôles, modération, chat, réactions, captions, socket temps réel.
- **UI** : `TopBar` (share/chat/tune[mod]/flag/End Room[host]) ; bannière statut audio ; en-tête room (badges house/catégorie/visibilité/REC, titre éditable si mod, `RoomTimer` /scheduled, compteurs, 🗨️) ; partials `StageGrid`, `HandRaiseQueue`, `FollowedByListeners`, `FlatList` « Others » (overflow +N) ; `RoomActionBar` (Mute/Raise hand/Visible-Invisible/Invite/Leave) ; `ReactionsBar` ; `ExtCaptionsOverlay` ; sheets/modals `HostActionsSheet`, `ProfileActionSheet`, `RoomChatSidebar`, `RoomControlsSheet`, `TitleEditModal`.
- **Données** : `useRoom(id)` (`GET /rooms/:id`) ; `useHandRaises` (poll 10s si realtime off) ; `useRoomSocket` (joined/hand/role/mute/kick/ended) ; `useRoomAudio` (LiveKit, scores parole) ; `useExtCaptions` / `useLocalCaptionPublisher` ; mutations `useLeaveRoom` / `useRaiseHand` / `useLowerHand` / `useSetMute` / `useSetHidden` / `useEndRoom` / `useReportRoom` ; stores `useAuthStore`, `useCurrentRoomStore`, `roomAudioSession` (singleton mini-bar).
- **Actions** : mute (optimiste + LiveKit + rollback) ; raise/lower hand ; ghost ; tap participant → Host/Profile sheet ; promote ; End Room (Alert) ; Report ; Share (`app.chathouse.com/room/:id`) ; Invite → `InviteToRoom` ; Message → cross-tab ChatDetail ; Leave → clear+stop+ `leaveRoom` +goBack.
- **États** : Loader/EmptyState ; bannière audio (connecting/unsupported/error) ; join refusé → goBack+Alert ; events socket filtrés sur viewerId/roomId (garde `cancelled`) ; rôles dérivés `viewerIsHost` / `canModerate` / `canSpeak`.
- **Navigation** : depuis RoomFeed/CreateRoom(`replace`)/Explore/Maps/deep-link `room/:id` → InviteToRoom, cross-tab ChatDetail ; back = mini-bar persiste.
- **Accès** : authentifié + autorisation serveur de join (public/social/closed/ban).

13. CreateRoomScreen

- **Rôle & contexte** : Création de room (live ou planifiée), en modal.
- **UI** : `Input` titre (3–80) / description (≤200) ; 3 `VisibilityRow` (Open/Social/Closed) ; `HouseChip` ; `TopicChip` (max 5) ; recherche co-hosts + `CoHostSlot` (max 5) ; toggle Schedule (presets / `DateTimePickerInline`) ; toggle Recording ; `Button` « Start room ».
- **Données** : `useCreateRoom()` (`POST /rooms`, `visibilityToBackend`) ; `useHouses('mine')` ; recherche co-hosts `searchService.users` (`GET /search?type=users`, debounce 250ms).
- **Actions** : édition locale ; `handleStart` → calcule `scheduledFor` → `createRoom.mutateAsync` → live : `replace('Room')` / planifiée : `goBack`.
- **États** : bouton désactivé si titre <3/>80 ou pending ; caps 5 topics / 5 co-hosts.
- **Navigation** : RoomFeed FAB / deep-link `room/new` → Room ou retour.
- **Accès** : authentifié.

14. InviteToRoomScreen

- **Rôle & contexte** : Inviter des utilisateurs dans une room existante (param `roomId`).
- **UI** : `Input` recherche débouncée, compteur, `FlatList` candidats (Avatar + checkbox), CTA « Envoyer (N) ».
- **Données** : `searchService.users` (debounce 250ms) ; `useInviteToRoom()` (`POST /rooms/:id/invite`).
- **Actions** : toggle (cap `MAX_INVITEES=50`) ; Envoyer → `invite.mutate` → Alert + goBack.
- **États** : Loader (recherche) ; EmptyState (vide/aucun résultat) ; CTA désactivé si 0 / pending.
- **Navigation** : RoomScreen → goBack.
- **Accès** : authentifié (tout participant).

15. ReplaysScreen

- **Rôle & contexte** : Replays audio récents (rooms enregistrées) lisibles in-app.
- **UI** : `FlatList` de cartes (titre, host · date, lecteur `ReplayPlayer` play/pause/progress).
- **Données** : `useRecentReplays()` → `recordingService.recent()`. Pas de mutation.
- **Actions** : lecture `ReplayPlayer` ; back.
- **États** : Loader ; EmptyState. Pas de scrub/transcript.
- **Navigation** : RoomFeed (play-circle) → goBack.
- **Accès** : authentifié (lecture seule).

4. Événements (1)

16. EventsScreen

- **Rôle & contexte** : Rooms planifiées (À venir + Mes events), RSVP + reschedule/cancel host.
 - **UI** : 2 `TabPill` (upcoming/mine) ; `FlatList` de `EventCard` (host, titre, schedule + temps relatif, compteur RSVP, bouton RSVP/Annuler, `ExtCalendarExportButton`.ics, boutons host Reschedule/Cancel) ; `Modal` reschedule (`DateTimePickerInline`).
 - **Données** : `useUpcomingEvents()` (`GET /rooms?filter=upcoming`), `useMyEvents()` (`GET /rooms/events/mine`), `useRsvp()` / `useCancelRsvp()` (`POST|DELETE /rooms/:id/rsvp`), `eventsApi.cancel / reschedule` ; `useAuthStore` (isHost).
 - **Actions** : switch tab ; toggle RSVP (anti-double-submit) ; host Cancel (Alert + refetch) ; host Reschedule (Modal → refetch).
 - **États** : Loader / EmptyState (empty/emptyMine) ; pull-to-refresh ; boutons disabled si mutating.
 - **Navigation** : RoomFeed (event) ; reste sur l'écran.
 - **Accès** : authentifié ; actions host si `hostId===viewerId` .
-

5. Clubs / « Houses » (6)

17. HouseListScreen

- **Rôle & contexte** : Liste des houses, onglets « My Houses » / « Discover ».
- **UI** : Header, `TabToggle` (mine/discover), `FlatList` de `HouseRow` (Avatar, nom, catégorie+emoji, membersCount), FAB « + » animé.
- **Données** : `useHouses(tab)` → `GET /clubs?filter=mine|discover`.
- **Actions** : row → `HouseDetail` ; FAB → `CreateHouse` ; pull-to-refresh ; switch onglet.
- **États** : Loader ; EmptyState (error). (Pas d'EmptyState liste vide dédié ; recherche club retirée.)
- **Navigation** : depuis Profil/Réglages → `HouseDetail`, `CreateHouse`.
- **Accès** : authentifié.

18. HouseDetailScreen

- **Rôle & contexte** : Fiche club : cover, stats, CTA d'adhésion contextuel, inbox demandes (admin SOCIAL), rooms, membres + rôles.
- **UI** : Header (settings si admin, more-vert) ; `FlatList` membres avec header riche (cover, avatar, description, règles, stats, CTA dynamique Invite/« Sur invitation »/« Request to join »/« Rejoindre ») ; inbox Approve/Decline ; sections rooms live/upcoming/past ; rows membres (badge rôle + more si gérable).
- **Données** : `useHouse(id)` (`GET /clubs/:id`) ; `useHouseRooms(id, live|upcoming|past)` ; `clubMetaApi.get` (`GET /ext/club-meta/:id`) ; `clubReqApi.list` (inbox, si `canManageRoles` && `isSocial`).
- **Actions** : Rejoindre OPEN `useJoinHouse` (`POST /clubs/:id/join`) ; SOCIAL `useExtJoinHouse.join` (`POST /ext/clubreq/:id/request`) ; approve/decline ; `useLeaveHouse` (masqué owner) ; `useSetMemberRole` / `useRemoveMember` (Alert) ; more-vert → Share/Invite/Quitter ; settings → `ManageHouse` ; Invite → `InviteMember` ; room → `Room`.
- **États** : Loader / EmptyState ; CTA selon privacy + `isJoinedByMe` ; owner ne quitte pas (CLUB_005) ; admin ne gère ni soi ni owner.
- **Navigation** : `HouseList/Notifications(house_invite)/Profil` → `Room`, `InviteMember`, `ManageHouse`.
- **Accès** : authentifié ; settings/inbox/rôles si admin ; inbox si SOCIAL.

19. CreateHouseScreen

- **Rôle & contexte** : Création d'une house.
- **UI** : upload icône (dashed/preview), `Input` nom (≤30) / description (≤200) / règles (≤2000), 3 `PrivacyRow` (Open/Social/Private), bouton « Create House ».
- **Données** : `useCreateHouse()` (`POST /clubs`, `PRIVACY_TO_DB`) ; `mediaService.uploadAvatar` ; `launchImageLibrary`.
- **Actions** : pick icône ; submit → upload puis `createHouse.mutateAsync` → `goBack`.
- **États** : `canCreate = nom≥2 && !pending && !uploading` ; spinner ; Alert erreur.
- **Navigation** : `HouseList` (FAB) → `goBack`.
- **Accès** : authentifié.

20. ManageHouseScreen

- **Rôle & contexte** : Édition d'une house + suppression (owner).
- **UI** : `Input` nom/description/règles, 3 `PrivacyRow`, « Save changes », et si owner « Danger zone » + « Delete house ».
- **Données** : `useHouse(id)` (pré-remplissage unique) ; `useUpdateHouse` (`PATCH /clubs/:id`) ; `useDeleteHouse` (`DELETE /clubs/:id`) ; `useAuthStore` (isOwner).
- **Actions** : save → `updateHouse.mutate` → `goBack` ; delete → Alert → `deleteHouse.mutate` → `HouseList`.
- **États** : Loader / EmptyState ; `canSave = nom≥2 && !mutating` ; suppression owner only.
- **Navigation** : `HouseDetail` (settings, si admin) → `goBack` / `HouseList`.
- **Accès** : admin ; delete owner.

21. HouseInvitationScreen

- **Rôle & contexte** : Acceptation d'une invitation (deep-link / notif CLUB_INVITE).
- **UI** : Avatar 120, nom, nb membres, code tronqué (8), boutons « Accept invitation » / « Decline ».
- **Données** : params {`houseId`, `inviteToken`} ; `useHouse(id)` ; `useAcceptInvitation` (`POST /clubs/:id/accept`, token non transmis).
- **Actions** : accept → `accept.mutateAsync` → `replace('HouseDetail')` ; decline/back → `goBack`.
- **États** : Loader / EmptyState ; bouton accept loading ; copie house-centric (inviteur absent).
- **Navigation** : deep-link/route `HouseInvitation` → `HouseDetail`.
- **Accès** : authentifié + `houseId/token`.

22. InviteMemberScreen

- **Rôle & contexte** : Inviter dans une house par recherche ou lien partageable.
- **UI** : bandeau lien (`app.chathouse.com/invite/:houseId` + copier), `Input` recherche, `FlatList` `UserRow` (bouton Invite/Invited).
- **Données** : `useSearchUsers(q)` (`GET /users/search`, debounce 250ms) ; `useInviteToHouse` (`POST /clubs/:id/invite`).
- **Actions** : copier lien → Clipboard + Share ; inviter → `inviteToHouse.mutate` (« Invited » on success, anti-double-submit).
- **États** : Loader ; EmptyState (vide/sans résultat) ; Alert échec.
- **Navigation** : `HouseDetail` / modal `InviteMemberModal` → `goBack`.
- **Accès** : authentifié.

6. Carte (1)

23. MapsScreen

- **Rôle & contexte** : Carte OSM des followers en ligne géolocalisés en temps réel (onglet MapsTab).
 - **UI** : `MapView` plein écran (tiles OSM/Voyager) ; `Marker` user (`UserLocationPulse`) + followers (`UserMapMarker`) ; `MapSearchBar` ; boutons `MyLocationButton` + `GhostModeToggle` ; `FollowerMiniCard` ; `MapTopAppBar`.
 - **Données** : `useCurrentLocation()` (permission/coords, geolocation watch 30s/25m, consentement GDPR) ; `useLocationBroadcast` (socket `maps:update-location`, sauf Ghost) ; `useFollowersOnMap()` (`GET /maps/followers` + deltas socket `maps:user-moved` / `user-offline`) ; `useGhostModeStore`. Mock si realtime off.
 - **Actions** : recherche (filtre local) ; tap pin → `animateToRegion` ; recenter ; MiniCard → Join (cross-tab Room) / Message (cross-tab ChatDetail) ; toggle Ghost.
 - **États** : EmptyState (permission denied/disabled) ; Loader (« Locating you », fallback Dakar à 8s) ; auto-center 1er fix ; temps réel socket.
 - **Navigation** : onglet Maps (deep-link `map`) → cross-tab Room / ChatDetail.
 - **Accès** : authentifié + permission localisation. Ghost coupe le broadcast.
-

7. Découverte (1)

24. ExploreScreen

- **Rôle & contexte** : Recherche unifiée (users/clubs/rooms/topics) + feed tendances.
 - **UI** : Header (« Browse topics » → TopicExplorer) ; `Input` recherche (debounce 200ms) ; `FilterChip` facettes (langues fr/en/ar/es + catégories) ; vues `RecentSearchesView` / `SearchResultsView` / `ExploreFeedView`.
 - **Données** : `useExplore()` (feed) ; `useSearch(q)` (`GET /search?type=all`) ; `useTopicSearch(q)` (`topicsApi.flat`) ; `useExtSearchHistory()` ; `useExtSearchRooms({q,language,topic,liveOnly})`.
 - **Actions** : saisie → bascule feed/résultats ; submit → `history.commit` ; toggle facettes ; résultat → Room/HouseDetail/Profile/TopicExplorer ; clear historique.
 - **États** : Loader / EmptyState (searchEmpty) ; feed pull-to-refresh ; recherches récentes si focus+vide.
 - **Navigation** : RoomFeed (search) → Room, HouseDetail, Profile, TopicExplorer.
 - **Accès** : authentifié.
-

8. Notifications (1)

25. NotificationsScreen

- **Rôle & contexte** : Centre de notifications, filtres, mark-read, swipe-delete, deep-link par type.
 - **UI** : Header (sous-titre « X non lues », « Tout marquer lu ») ; `TabPill` (all/rooms/social/clubs) ; `FlatList` de `NotificationRow` dans `Swipeable` (delete) : icône par `kind`, message, date, point bleu si non lu.
 - **Données** : `useNotifications(filter)` (`GET /notifications?filter=`, `typeToKind`) ;
`useMarkNotificationRead` / `useMarkAllNotificationsRead` / `useRemoveNotification` ; `useNotificationSocket` (`notification:new` / `count`).
 - **Actions** : tap → mark-read + deep-link (follow/wave/new_message → Profile ; house_invite → HouseDetail ; room_* → Room) ; swipe delete ; mark-all ; pull-to-refresh.
 - **États** : Loader / EmptyState ; `unreadCount` pilote sous-titre+bouton ; live via socket. (new_message → profil expéditeur, DM hors RoomStack.)
 - **Navigation** : RoomFeed (cloche) → Profile, HouseDetail, Room.
 - **Accès** : authentifié (canal `user:<id>`).
-

9. Profil & Identité (4)

26. ProfileScreen

- **Rôle & contexte** : Profil self ou tiers (identité, stats, actions sociales, events/replays, « qui a vu » premium, sections self). Enregistré dans RoomStack ET SettingsStack.
- **UI** : `ProfileHeaderBar` (edit si self / share / more si tiers) ; `ProfileIdentity` (avatar+online, vrai nom, @username copiable, Joined, Nominated by, bio, liens Twitter/Insta) ; `ExtBadgesRow` + `ExtProfileLinks` ; `ProfileStats` ; `ProfileActionButtons` (Follow + wave, tiers) ; bannière live ; sections Events/Replays/« Qui a vu » (self) / `SelfSections`.
- **Données** : `useMe()` + `useAuthStore` (myId) ; `useProfile(userId)` (GET /users/:id) ; `useFollow` / `useUnfollow` / `useWave` / `useBlock` / `useReport` ; `useHouses('mine')` ; `useMyRoomHistory(10)` ; `useUserUpcomingEvents` / `useUserReplays` ; `useProfileViewers(isSelf)` (premium, `retry:false`).
- **Actions** : edit → `EditProfile` ; share ; copy username (haptic) ; follow toggle ; wave (USER_005 rate-limit) ; report (Alert raisons) ; block (Alert) ; stats → Followers ; sections → Room/Replays/HouseList/HouseDetail.
- **États** : Loader / EmptyState ; `isSelf` masque actions sociales + montre sections self ; « qui a vu » premium-gate (🔒) / empty / liste.
- **Navigation** : tabs/notifications/HouseDetail/Followers → `EditProfile`, Followers, Room, Replays, HouseList, HouseDetail.
- **Accès** : authentifié ; sections self + viewers réservés au propriétaire (viewers = premium serveur).

27. EditProfileScreen

- **Rôle & contexte** : Édition du profil courant.
- **UI** : avatar + `photo-camera` ; `Input` firstName/lastName, displayName (≤40), username (@), bio (≤150), twitter/instagram ; ligne DM privacy (Alert) ; « Save changes ».
- **Données** : `useMe()` (pré-remplit) ; `useUpdateProfile` (PATCH /users/me + PATCH /users/me/username si changé) ; `mediaService.uploadAvatar` ; `launchImageLibrary` ; `usernameFormSchema`.
- **Actions** : pick image ; DM privacy → Alert (everyone/followers/mutual/nobody) ; save → upload puis `updateProfile.mutateAsync` → goBack.
- **États** : Loader ; `busy` (upload+PATCH) ; `canSave = displayName≥2 && username valide && !busy` ; Alert erreur.
- **Navigation** : ProfileScreen (self) → goBack.
- **Accès** : authentifié, self (/users/me).

28. FollowersScreen

- **Rôle & contexte** : Abonnés / abonnements d'un utilisateur, onglets + follow inline.
- **UI** : `TabToggle` (Followers/Following), `FlatList` de `UserRow` (bouton Follow/Following, loader par ligne).
- **Données** : params {userId, initialTab} ; `useFollowers` / `useFollowing` (GET /follow/:id/followers|following) ; `useFollow` / `useUnfollow`.
- **Actions** : switch onglet ; toggle ligne (Alert erreur).
- **États** : Loader / EmptyState (error/vide) ; `pendingId` (spinner ciblé).
- **Navigation** : ProfileScreen (compteurs) → goBack.
- **Accès** : authentifié (self ou tiers).

29. TipHistoryScreen

- **Rôle & contexte** : Historique des pourboires (monétisation V7) envoyés/reçus.
- **UI** : `FlatList` de `TipRow` (icône sens, sent/received, contrepartie + date, montant `Intl.NumberFormat` signé).
- **Données** : `useQuery(['ext', 'payments', 'tips'])` → `paymentsApi.tipHistory()` (GET /ext/payments/tips, staleTime 30s).
- **Actions** : back ; pull-to-refresh.
- **États** : Loader / EmptyState (error/vide).
- **Navigation** : `SettingsTab/TipHistory` → goBack.
- **Accès** : authentifié (extension payments).

10. Messages / « Backchannel » (6)

30. MessagesScreen

- **Rôle & contexte** : Onglet Messages : conversations 1:1 + groupes.
- **UI** : Header (bouton « nouveau ») ; `FlatList` de `ConvoRow` ; `ListHeader` = `OnlineUsersList` (rend `null` faute de présence) + section Groups (`GroupRow`). Rows : avatar, nom, dernier message (🗣️ si vocal), timestamp relatif, badge non-lus.
- **Données** : `useConversations()` (`GET /chat/conversations`) ; `useGroups()` (`GET /groups`) ; `useAuthStore` (myId) ; `useChatSocket()` + `useGroupSocket()` (invalident sur `chat:message` / `chat:read` / `group:message`).
- **Actions** : row DM → `ChatDetail` ; groupe → `GroupChat` ; edit → `NewMessage` ; pull-to-refresh.
- **États** : Loader / EmptyState (error/vide) ; timestamps figés au render.
- **Navigation** : tab Messages → ChatDetail / GroupChat / NewMessage.
- **Accès** : authentifié.

31. ChatDetailScreen

- **Rôle & contexte** : Thread DM 1:1 (`conversationId` == id du pair == `receiverId`).
- **UI** : `ChatHeader` (avatar, nom, dot online, « typing... », call/more) ; `FlatList inverted` de `Bubble` (texte `ExtLinkifiedText` , vocal `VoiceMessageBubble` , `done-all`) + `DateSeparator` ; `ChatInputBar` (emoji 16, multiline, attach, send/mic) ↔ `VoiceRecordingBar` .
- **Données** : `useConversation(peerId)` ; `useConversationMessages(peerId)` (`GET /chat/:peerId`) ; `useTypingIndicator(peerId)` (`chat:typing`) ; `useVoiceMessage` ; mutations `useSendMessage` / `useSendVoiceMessage` / `useMarkConversationRead` / `useDeleteMessage` .
- **Actions** : envoi texte (`POST /chat/:id`) ; frappe → `notifyTyping` ; mic → record → `POST /chat/:id/voice` ; long-press son message → Alert → delete ; auto mark-read ; `call/more/attach` → Alert « Coming soon ».
- **États** : Loader ; typing (clear 4s) ; vocal `VoiceRecordingBar` ; `isOnline` figé `false` (présence non câblée) ; erreurs via toast.
- **Navigation** : MessagesList/NewMessage/OnlineUsersList → goBack ; tab bar masquée.
- **Accès** : authentifié ; DM gaté follow mutuel (403 CHAT_004).

32. NewMessageScreen

- **Rôle & contexte** : Sélecteur de destinataires : 1 → DM, ≥2 → groupe.
- **UI** : `Input` recherche (autoFocus), `FlatList` résultats (coche), `Button` flottant dynamique (« Message » / « Create group · N »).
- **Données** : `searchService.users(q,20)` (debounce 250ms) ; `useCreateGroup()` (`POST /groups`).
- **Actions** : toggle ; start : 1 → `replace('ChatDetail')` ; ≥2 → `createGroup.mutate` → `replace('GroupChat')` .
- **États** : EmptyState (hint/noResults) ; `createGroup.isPending` .
- **Navigation** : MessagesScreen → ChatDetail / GroupChat (replace).
- **Accès** : authentifié.

33. GroupChatScreen

- **Rôle & contexte** : Thread de groupe.
- **UI** : Header (titre + memberCount → infos) ; `FlatList inverted` (nom expéditeur sur reçus, vocal) ; barre saisie send/mic ↔ `VoiceRecordingBar` .
- **Données** : `useGroup(id)` ; `useGroupMessages(id)` ; mutations `useSendGroupMessage` / `useSendGroupVoice` / `useMarkGroupRead` ; `useVoiceMessage` ; `nameById` .
- **Actions** : envoi (`POST /groups/:id/messages` , draft restauré si erreur) ; vocal (`/voice`) ; auto mark-read ; header → `GroupInfo` .
- **États** : Loader ; `VoiceRecordingBar` ; `send.isPending` .
- **Navigation** : MessagesScreen/NewMessage → GroupInfo.
- **Accès** : authentifié, membre.

34. GroupInfoScreen

- **Rôle & contexte** : Paramètres d'un groupe.
- **UI** : icône groupe ; `Input` nom (≤80) + check ; section membres (« Add people », rows avec « remove » si owner) ; bouton « Leave group » (danger).
- **Données** : `useGroup(id)` ; `useRenameGroup` / `useRemoveGroupMember` / `useLeaveGroup` ; `isOwner` .
- **Actions** : save titre ; remove (Alert) ; leave (Alert → `popToTop`) ; Add people → `AddGroupMembers` .
- **États** : Loader ; save désactivé si inchangé/pending ; remove si `isOwner && !isMe` .
- **Navigation** : GroupChat → AddGroupMembers ; `popToTop` après leave.
- **Accès** : authentifié, membre ; rename/remove owner.

35. AddGroupMembersScreen

- **Rôle & contexte** : Ajouter des membres à un groupe.
- **UI** : `Input` recherche autoFocus ; `FlatList` (membres existants grisés/ `check` , autres toggables) ; `Button` « Add N ».
- **Données** : `useGroup(id)` (existingIds) ; `searchService.users(q,20)` ; `useAddGroupMembers()` (`POST /groups/:id/members`).
- **Actions** : toggle ; « Add N » → `addMembers.mutate` → goBack.
- **États** : EmptyState (hint/noResults) ; `isPending` .
- **Navigation** : GroupInfo → goBack.
- **Accès** : authentifié, membre.

11. Réglages (3)

36. SettingsScreen

- **Rôle & contexte** : Hub profil + paramètres (onglet Settings).
- **UI** : top-bar (Wave); hero avatar + stats; Edit/Wave/More; « Create House »; grille « Member of »; `ExtPremiumRow` (conditionnel); Tip history; entrée Godmode (conditionnelle); section Privacy (toggle crash-reporting, Policy, Terms, Export, Blocked, `ExtThemeToggle`); section Account (Invite friends, Notifications, Extensions, Delete account).
- **Données** : `useMe()`; `useHouses('mine')`; `useAdminWhoami()` (GET /admin/me, staleTime 60s, `retry:false`); `useAnalyticsConsentStore`; `useQuery(['ext','invite','link'])` (GET /ext/invites/link); `useAuthStore.signOut`.
- **Actions** : Edit → EditProfile; Wave → Followers; More → signOut; Create/View/tuiles → RoomsTab (CreateHouse/HouseList/HouseDetail); stats → Followers; toggle analytics (gate Sentry); routes Policy/Terms/DataExport/BlockedUsers/TipHistory/NotificationSettings/ExtSettings/DeleteAccount; Invite → Share (Alert si `remaining≤0`).
- **États** : pas de loader plein écran (valeurs par défaut); Godmode/Premium conditionnels.
- **Navigation** : SettingsTab (route initiale) → routes SettingsStack + cross-tab RoomsTab.
- **Accès** : authentifié; Godmode si rôle ≥ MODERATOR.

37. NotificationSettingsScreen

- **Rôle & contexte** : Préférences notifs : 9 toggles + fréquence + mute club/utilisateur.
- **UI** : `PrefRow` (Switch) ×9 (NOTIF_PREF_KEYS); `FrequencySelector` (infrequent/normal/frequent); `MuteRow` par club et par utilisateur muté.
- **Données** : `useNotifPrefs()` / `useUpdateNotifPrefs()` (GET|PATCH /users/me/notification-preferences, optimiste); `useNotifPrefsExt()` (GET /ext/notif-prefs) + `useSetNotifFrequency` / `useToggleMutedClub` / `useUnmuteUser`; `clubsListApi.myClubs()` (noms de clubs).
- **Actions** : toggle pref; tier; toggle club; unmute user (le mute se fait depuis le profil).
- **États** : Loader / EmptyState + Retry; rows disabled si pending; sections vides masquées.
- **Navigation** : SettingsScreen → goBack.
- **Accès** : authentifié.

38. BlockedUsersScreen

- **Rôle & contexte** : Comptes bloqués + déblocage.
- **UI** : `FlatList` de `BlockedRow` (Avatar + « Unblock » danger, loader par ligne).
- **Données** : `useBlockedUsers()` (GET /users/me/blocked); `useUnblock()` (DELETE /users/:id/block).
- **Actions** : Unblock → `unblock.mutate` (invalidation → disparaît; Alert erreur).
- **États** : Loader / EmptyState (error/vide); `pendingId`.
- **Navigation** : SettingsScreen → goBack.
- **Accès** : authentifié.

12. Confidentialité & Légal (4)

39. DataExportScreen

- **Rôle & contexte** : Export RGPD (art. 20) via la feuille de partage OS.
- **UI** : carte descriptive, « Export », feedback taille (Ko), « Copy » (opt-in), « Clear clipboard ».
- **Données** : `privacyService.exportMyData()` (`GET /users/me/export` , JSON brut, gardé en mémoire) ; `Clipboard` ; `Share` .
- **Actions** : Export → `Share.share` ; Copy → Clipboard ; Clear → vide.
- **États** : `busy` ; Alert erreur ; pas de file-export (expo absent).
- **Navigation** : SettingsScreen → DataExport (header masqué).
- **Accès** : authentifié.

40. DeleteAccountScreen

- **Rôle & contexte** : Suppression de compte (saisie « DELETE » + délai de grâce).
- **UI** : carte d'avertissement, étapes (export/rooms fermées/DM non rétractables), `TextInput` confirmation, « Delete » danger désactivé tant que ≠ « DELETE ».
- **Données** : `privacyService.requestDeletion()` (`POST /users/me/request-deletion`) ; `useAuthStore.signOut` .
- **Actions** : Delete → Alert → `requestDeletion()` → `signOut()` .
- **États** : `busy` ; `canDelete = saisie === DELETE` .
- **Navigation** : SettingsScreen → DeleteAccount → signOut (retour Auth).
- **Accès** : authentifié.

41. PrivacyPolicyScreen

- **Rôle & contexte** : Politique de confidentialité statique in-app (offline, versionnée au build).
- **UI** : `LegalDoc` (7 `LegalSection` , email `LegalEmail` `privacy@chathouse.app`).
- **Données** : aucune (i18n `privacy.policy.*`).
- **Actions** : aucune (scroll).
- **États** : aucun.
- **Navigation** : monté dans Auth (pré-auth depuis Landing) ET Settings.
- **Accès** : public + authentifié.

42. TermsScreen

- **Rôle & contexte** : Conditions d'utilisation statiques in-app.
 - **UI** : `LegalDoc` (8 `LegalSection`).
 - **Données** : aucune (i18n `privacy.terms.*`).
 - **Actions** : aucune.
 - **États** : aucun.
 - **Navigation** : monté dans Auth ET Settings.
 - **Accès** : public + authentifié.
-

13. Extensions (5)

43. ExtActivityFeedScreen

- **Rôle & contexte** : Flux d'activité (Module 12.7), filtres + live-prepend socket (réutilise `/notifications`).
- **UI** : header (« Mark all read »), 4 onglets (all/rooms/social/clubs), `FlatList` de `ActivityRow` (memo), pull-to-refresh, empty.
- **Données** : `activityApi.list(filter)` (`GET /notifications?filter=`) + `markRead / markAllRead` ; `useExtSocketAliases` (events `room_started_by_following / join_request / ping_user`, prepend `live-`, cap 200, dédup).
- **Actions** : filtre ; tap → `markRead` + `onTapItem?` (undefined en prod) ; mark all ; refresh.
- **États** : `ActivityIndicator` ; vide « No activity yet » ; erreur → garde liste stale.
- **Navigation** : **route orpheline** (`RoomsNavigator`, aucun `navigate('ActivityFeed')` ni deep-link).
- **Accès** : authentifié.

44. ExtSuggestedFollowsScreen

- **Rôle & contexte** : « People you may know » — dernière étape d'onboarding (Module 1.5), réutilisable.
- **UI** : `FlatList` de `UserRow` (memo : raison colorée, Follow → « Following »), pull-to-refresh.
- **Données** : `useExtSuggestions(30)` (`GET /ext/suggestions?limit=30`, staleTime 60s ; raisons `shared_interests/friends_of_friends/trending`).
- **Actions** : Follow → `onFollow` (en onboarding = `useFollow().mutate POST /follow/:id`, anti-doublon + rollback) ; tap → `onTapUser`.
- **États** : `ActivityIndicator` / vide ; « Following » désactivé.
- **Navigation** : via `SuggestedFollowsRoute` (`OnboardingNavigator`, route `SuggestedFollows`) ; footer Done/Skip → `completeOnboarding` (`PATCH /users/me/onboarding`) + redeem invite.
- **Accès** : onboarding (étape finale).

45. ExtTopicExplorerScreen

- **Rôle & contexte** : Explorateur de 150+ topics (2 panneaux + recherche + tendances).
- **UI** : recherche (liste plate) ou défaut (strip tendances + panneaux catégories/sous-topics).
- **Données** : `useExtTopicsTree()` (`GET /ext/topics`, staleTime 24h) ; `useExtTopicsFlat(q)` ; `useExtTopicsTrending()`.
- **Actions** : recherche ; tap catégorie ; tap topic → `onSelectTopic?(slug)` (prop).
- **États** : `ActivityIndicator` ; panneau droit vide « Pick a category ».
- **Navigation** : `RoomsNavigator` (route `TopicExplorer`), depuis Explore (`goTopics`). Monté sans `onSelectTopic` → sélection sans effet de nav.
- **Accès** : authentifié.

46. ExtSettingsScreen

- **Rôle & contexte** : Paramètres extensions (apparence, qualité audio, moteur audio, confidentialité).
- **UI** : Appearance (`ExtThemeToggle`) ; Audio quality (3 tiers radio standard/high/music) ; Audio engine (3 Switch : spatial audio, noise suppression, drop-in) ; Privacy (3 Switch : private profile, allow waves, show on map).
- **Données** : `audioApi.get/update` (`GET|PATCH /ext/audio`) ; `privacyApi.get/update` (`GET|PATCH /ext/privacy`). Updates optimistes + rollback.
- **Actions** : tap tier / switches → mutations partielles (Alert sur échec).
- **États** : `ActivityIndicator` ; défauts si null.
- **Navigation** : `SettingsNavigator` (route `ExtSettings`), depuis Settings → Extensions.
- **Accès** : authentifié.

47. ExtPlaygroundScreen

- **Rôle & contexte** : Banc de QA dev rendant les composants d'extension V1–V15 en isolation.
- **UI** : sections de démo (linkified text, deep-links sociaux, available-people, upcoming, network bars, calendar export, share sheet, reaction picker, captions overlay, validateur d'intérêts, back-to-room, chat reactions, badges) + champs de test.
- **Données** : surtout mocks ; `useExtWave`, `validateInterests` ; `ExtBadgesRow / ExtAvailablePeopleStrip` font des fetch réels.
- **Actions** : ouvrir sheets/pickers, wave, deep-links, éditer inputs.
- **États** : statut wave / validation intérêts / modals.
- **Navigation** : **non câblé** (exporté seulement, aucune route).
- **Accès** : aucun en prod (dev/QA).

14. Administration / « Godmode » (6) — rôle ≥ MODERATOR

Infra : `adminService (/admin/*)`, `useAdmin` (queries/mutations), `types/admin.types` (`ROLE_RANK`, `isAtLeast`), `AdminHeader`, `promptForReason`, sous-système d'impersonation (`ImpersonationBanner` global). Entrée = tuile Godmode dans Settings (si rôle ≥ MODERATOR). **Chaque endpoint `/admin/*` re-vérifie le rôle serveur** (`whoami retry:false`).

48. AdminHomeScreen

- **Rôle & contexte** : Tableau de bord Godmode : KPI globaux + tuiles + exports CSV (SUPER_ADMIN).
- **UI** : `AdminHeader` (« Connected as {role} ») ; 6 `KpiCard` (Users/online, Live, Reports, Suspended, New 24h, Messages 24h) ; `NavTile` (Users, Reports [badge open], Rooms [si ADMIN], Audit [si SUPER_ADMIN]) ; section « CSV Exports » (conditionnelle).
- **Données** : `useAdminWhoami()` (`GET /admin/me`) ; `useAdminStats()` (`GET /admin/stats`, `refetchInterval 30s`).
- **Actions** : tuiles → AdminUsers/Reports/Rooms/AuditLog ; export → `adminService.exportCsv(kind)` (`GET /admin/export/{kind}.csv`) → Clipboard + Share.
- **États** : Loader / EmptyState ; `exporting` désactive boutons ; gardes `canSeeAuditLog` (SUPER) / `canForceEnd` (ADMIN).
- **Navigation** : Settings (tuile Godmode) → AdminUsers/Reports/Rooms/AuditLog.
- **Accès** : MODERATOR ; Rooms = ADMIN ; Audit + CSV = SUPER_ADMIN.

49. AdminUsersScreen

- **Rôle & contexte** : Annuaire/recherche utilisateurs, filtre rôle, liste curseur.
- **UI** : `Input` recherche ; `filterChip` (ALL/USER/MOD/ADMIN/SUPER) ; `FlatList` de `UserRow` (Avatar+online, badge rôle coloré, badges Suspended/Deleted) ; footer pagination ; pull-to-refresh.
- **Données** : `useAdminUsersInfinite({q,role,limit:50})` (`GET /admin/users`, curseur, debounce 250ms).
- **Actions** : recherche/filtre → `refetch` ; row → `AdminUserDetail` ; `onEndReached` → `fetchNextPage`.
- **États** : Loader / EmptyState (error/vide) ; footer `isFetchingNextPage`.
- **Navigation** : AdminHome → AdminUserDetail.
- **Accès** : MODERATOR.

50. AdminUserDetailScreen

- **Rôle & contexte** : Fiche utilisateur + actions de modération (gardées par hiérarchie).
- **UI** : héros (avatar+role+suspension) ; `Field` infos (email, tél, inscription, dernière connexion, followers) ; sections Suspension (presets 1h/24h/7d/perm ou Unsuspend), Rôles (SUPER), Investigation/Impersonate (SUPER), Danger/Delete (SUPER) ; sinon encart « noActionPerm ».
- **Données** : `useAdminWhoami()` ; `useAdminUser(userId)` (`GET /admin/users/:id`).
- **Actions** (Alert confirm + `promptForReason`) : `useSetUserRole` (`PATCH .../role`) ; `useSuspendUser` (`POST .../suspend`) ; `useUnsuspendUser` ; `useImpersonationStore().start` (`POST .../impersonate` → `popToTop` + bannière) ; `useDeleteUser` (`DELETE ...` → `goBack`).
- **États** : Loader / EmptyState ; `canActOnTarget = rang(moi) > rang(cible)` ; rôle/impersonate/delete = SUPER ; masqués si `deletedAt`.
- **Navigation** : AdminUsers → `goBack/popToTop`.
- **Accès** : MODERATOR (rang strict > cible) ; rôle/impersonate/delete = SUPER_ADMIN.

51. AdminRoomsScreen

- **Rôle & contexte** : Rooms (live par défaut) + terminaison forcée.
- **UI** : avis explicatif ; `FlatList` de `RoomRow` (host, titre, participants, badge LIVE, bouton close/« Ended », busy par room) ; pull-to-refresh.
- **Données** : `useAdminRooms({live:true})` (`GET /admin/rooms?live=true`) ; `useForceEndRoom`.
- **Actions** : force-end → `promptForReason` → `POST /admin/rooms/:id/force-end` (notifie + coupe LiveKit) ; invalide rooms+stats.
- **États** : Loader / EmptyState ; bouton désactivé si non-live/busy.
- **Navigation** : AdminHome (tuile Rooms, ADMIN+).
- **Accès** : ADMIN.

52. AdminReportsScreen

- **Rôle & contexte** : File de modération des signalements (USER/ROOM), onglets + résolution.
- **UI** : `tabPill` (Open/Resolved/All) ; `FlatList` de `ReportRow` (badge type-motif, cible user/room, rapporteur+date, détails, boutons Dismiss/Resolve si ouvert) ; pull-to-refresh.
- **Données** : `useAdminReports({status})` (`GET /admin/reports?status=`) ; `useResolveReport`.
- **Actions** : onglet → `refetch` ; Dismiss/Resolve → Alert → `POST /admin/reports/:id/resolve {outcome}` ; invalide reports+stats.
- **États** : Loader / EmptyState (selon onglet).
- **Navigation** : AdminHome (tuile Reports).
- **Accès** : MODERATOR.

53. AdminAuditLogScreen

- **Rôle & contexte** : Journal lecture seule des actions privilégiées.
- **UI** : `FlatList` de `Row` (icône action, acteur → cible avec avatars, métadonnées `from→to` /reason/until, date + IP). Pas de filtre exposé.
- **Données** : `useAdminAuditLog({limit:100})` (`GET /admin/audit-log?limit=100`, une page de 100).
- **Actions** : pull-to-refresh (aucune mutation).
- **États** : Loader / EmptyState.
- **Navigation** : AdminHome (tuile Audit, SUPER_ADMIN).

- **Accès** : SUPER_ADMIN.
-

Constats notables (factuels)

- **Routes orphelines / non câblées** : `WaitlistScreen` (deep-link seul), `ExtActivityFeedScreen` (aucun point d'entrée), `ExtPlaygroundScreen` (jamais monté, dev/QA), `ExtTopicExplorerScreen` (monté sans `onSelectTopic` → sélection sans nav).
- « **Coming soon** » dans ChatDetail : appel vocal, pièces jointes, options de conversation.
- **Présence en ligne** non branchée (`OnlineUsersList` rend `null` ; `isOnline` figé `false`).
- **Mocks résiduels** : `CURRENT_USER` sert de fallback d'identité dans Messages/ChatDetail.
- **Sécurité admin** : le masquage client (`isAtLeast`) est doublé d'un contrôle serveur sur chaque `/admin/*` .

Fin du manuel détaillé — 53 écrans. Document interne ChatHouse.